

Reviewer Pack — Minimal Order of Quadratic Forms and Communication Complexit...

Entry b833db58bb8e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 15:50:43 UTC

Minimal Order of Quadratic Forms and Communication Complexity Rank Variance

Entry ID: b833db58bb8e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 15:50:43 UTC

1. Conjecture

Field A (mathematical branch): Quadratic Form Theory **Field B** (complexity object): Communication Complexity

Statement:

For every instance φ of a communication problem, the minimal order of a quadratic form that is invariant under the action of the symmetry group of φ is equal to $O(1/|\varphi|^3)$ times the variance of its matrix representation, i.e., $\min_order(Q(\varphi)) = \Theta(1/|\varphi|^3 \times \text{Var}(M(\varphi)))$

Rationale (proposer's reasoning):

Quadratic forms provide a rich algebraic structure that can capture symmetries in complex systems. Communication complexity ranks the difficulty of solving communication problems, and the variance of its matrix representation measures the spread of the rank values across instances. The conjecture suggests that the algebraic structure of quadratic forms can be used to understand the complexity of communication problems.

Taxonomy category: QUADRATIC_FORM_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1a99ee70984fcbf0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for a set of random communication complexity instances φ with n variables, the correlation coefficient between $\min_order(Q(\varphi))$ and $\text{Var}(M(\varphi))$ is ≥ 0.8 across at least 100 seeds, where $\min_order(Q(\varphi))$ is computed as $O(1/|\varphi|^3)$ times the variance of $M(\varphi)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_communication_instance(n):
        # Generate a random communication instance matrix  $M(\varphi)$ 
        return [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]

    def variance(matrix):
        n = len(matrix)
        mean = sum(sum(row) for row in matrix) / (n * n)
        return sum((x - mean) ** 2 for row in matrix for x in row) / (n * n)

    def min_order(M):
        # Placeholder function to compute the minimal order of a quadratic form
        # This is a dummy implementation and should be replaced with actual computation
        return variance(M) / (len(M) ** 3)

    instances_tested = 0
    correlation_coefficient_sum = 0.0
    n_max = 0

    for _ in range(100):
        n = random.choice([5, 10, 15, 20, 30, 40])
        M = generate_communication_instance(n)
        min_order_value = min_order(M)
        instances_tested += 1
        if n > n_max:
            n_max = n

        # Calculate the correlation coefficient (this is a dummy implementation)
        # In practice, this would involve comparing min_order with other metrics
        correlation_coefficient_sum += min_order_value

    mean_correlation_coefficient = correlation_coefficient_sum / instances_tested
```

```

return {
    "metric_name": "correlation_coefficient",
    "metric_value": mean_correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": False, # This is a dummy implementation
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ping_undefined'}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.05785569487771486, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.035442393388659205, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.04315816324669304, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.07318424024963796, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.061936137292491776, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.062161343676708836, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.05459980389271068, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.04836700469545012, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.04545560418487169, 'instances_tested': 1000000}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f4fd6092.py", line 80, in <module>
    first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]), None)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture’s validity. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12283
2	preregistration	ollama_remote	glm4:latest	0	0	10010
3	novelty	ollama_remote	glm4:latest	0	0	10340
4	novelty	ollama_remote	glm4:latest	0	0	8502
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26843
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7254
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10502
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8577
9	judge	ollama_remote	glm4:latest	0	0	12277

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106588 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b833db58bb8e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b833db58bb8e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b833db58bb8e.tar.gz` (if generated)